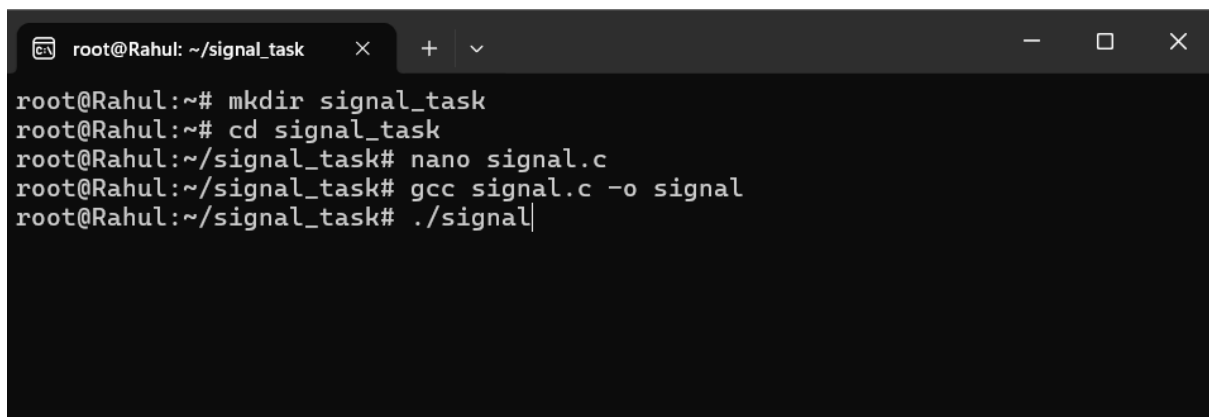


Practical-6

Task - 2:

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.



```
root@Rahul: ~/signal_task
root@Rahul:~# mkdir signal_task
root@Rahul:~# cd signal_task
root@Rahul:~/signal_task# nano signal.c
root@Rahul:~/signal_task# gcc signal.c -o signal
root@Rahul:~/signal_task# ./signal
```

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig) {
    if (sig == SIGINT) {
        printf("\nReceived SIGINT (Ctrl+C)\n");
        printf("Program is still running...\n");
    }
    else if (sig == SIGTERM) {
        printf("\nReceived SIGTERM\n");
        printf("Program terminating...\n");
        exit(0);
    }
}
```

```

    else if (sig == SIGUSR1) {
        printf("\nReceived SIGUSR1\n");
        printf("Asynchronous event handled successfully.\n");
    }
}

int main() {
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);
    printf("POSIX Signal Handling Program\n");
    printf("Process ID: %d\n", getpid());
    printf("Press Ctrl+C to generate SIGINT.\n");
    printf("Use kill -USR1 %d to generate SIGUSR1.\n", getpid());
    printf("Use kill -TERM %d to generate SIGTERM.\n\n", getpid());

    while (1) {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}

```

Output:

```
root@Rahul: ~/signal_task
root@Rahul:~# mkdir signal_task
root@Rahul:~# cd signal_task
root@Rahul:~/signal_task# nano signal.c
root@Rahul:~/signal_task# gcc signal.c -o signal
root@Rahul:~/signal_task# ./signal
POSIX Signal Handling Program
Process ID: 445
Press Ctrl+C to generate SIGINT.
Use kill -USR1 445 to generate SIGUSR1.
Use kill -TERM 445 to generate SIGTERM.

Program is running...
Program is running...
Program is running...
Program is running...
^C
Received SIGINT (Ctrl+C)
Program is still running...
Program is running...
Program is running...
Program is running...
Program is running...
|
```

```
root@Rahul: ~/signal_task
root@Rahul:~/signal_task# ./signal
POSIX Signal Handling Program
Process ID: 515
Press Ctrl+C to generate SIGINT.
Use kill -USR1 515 to generate SIGUSR1.
Use kill -TERM 515 to generate SIGTERM.

Program is running...
44Program is running...
5Program is running...
^C
Received SIGINT (Ctrl+C)
Program is still running...
Program is running...
445
Program is running...
Program is running...
Program is running...
Program is running...

root@Rahul:~# kill -USR1 445
root@Rahul:~# kill -TERM 445
root@Rahul:~# kill -USR1 445
-bash: kill: (445) - No such process
root@Rahul:~#
```

OBSERVATION: The program successfully captured and handled the SIGINT, SIGTERM, and SIGUSR1 signals using POSIX signal handling. A signal handler was registered for each signal, and when a signal was sent to the running process, the corresponding handler was executed and the appropriate message was displayed. The experiment demonstrated asynchronous event handling, where signals can be received by a process at any time during its execution. Thus, the role of signal handlers in responding to and managing asynchronous events was successfully demonstrated.